

DAT402 - Project1

Emily Chiu & Willis Dongmo

2026-04-07

R Markdown

Introduction and Objective

This project analyzes a student academic performance dataset containing 10,000 observations and multiple features related to study habits, attendance, academic background, and student context. The dataset also includes three outcome variables: GPA, grade category (A–F), and pass/fail status. The dataset is available here : <https://www.kaggle.com/datasets/sadiajavedd/student-academic-performance>

The primary objective of this project is to predict whether a student passes or fails, using the existing pass/fail variable as the binary target. This problem is framed as a supervised classification task, with the broader goal of identifying the key factors associated with student success and building models that can support early academic intervention.

To achieve this, two machine learning methods presented in class will be compared:

Logistic Regression with Regularization, chosen as an interpretable baseline model that estimates the probability of passing while reducing overfitting through regularization. Random Forest, selected as a flexible ensemble model capable of capturing non-linear relationships and feature interactions.

These two methods provide a meaningful comparison between a linear probabilistic classifier and a more complex tree-based approach.

The predictor variables include student-related academic and behavioral features, while GPA and grade category are excluded from the predictors to avoid data leakage, since they are strongly related to the target outcome.

The analysis will proceed through the following stages:

-Data loading and initial structure inspection -Dataset cleaning and preparation -Brief exploratory data analysis (EDA) -Train/test split -Model training and hyperparameter tuning -Performance evaluation using classification metrics -Final comparison of Logistic Regression and Random Forest

Data Loading and Initial Structure Inspection

This section loads the dataset and provides an initial overview of its structure, including variable names, data types, and dimensions.

```
library(here)
```

```
## here() starts at C:/Users/bruce/Downloads
```

```
dfi <- read.csv(here("student_exam_performance_dataset (1).csv"))
dim(dfi)
```

```
## [1] 10000    23
```

```
head(dfi)
```

```
##   student_id gender age parental_education family_income internet_access
## 1   S00001   Male  17      High School      Medium          Yes
## 2   S00002 Female  18      High School      Low            Yes
## 3   S00003   Male  17      High School      Medium          No
## 4   S00004   Male  18      Bachelor       Medium          Yes
## 5   S00005   Male  18      Bachelor       Medium          Yes
## 6   S00006 Female  17      Master        Medium          Yes
##   study_environment study_hours_per_day attendance_rate sleep_hours
## 1             Quiet             2.98             96.5         6.05
## 2             Quiet             4.45             95.7         6.96
## 3             Quiet             3.75             76.0         7.02
## 4             Quiet             2.03             72.6         6.23
## 5             Quiet             5.14             87.3         8.54
## 6             Quiet             1.16             92.0         8.16
##   social_media_hours assignment_completion_rate participation_score
## 1                0.1                80.5                68.7
## 2                2.9                70.9                92.6
## 3                2.4                77.6                45.8
## 4                3.5                63.5                72.9
## 5                2.1                71.8                55.7
## 6                2.6                100.0                76.3
##   online_courses_completed tutoring math_score reading_score writing_score
## 1                      1      Yes      42.8         62.4         54.8
## 2                      0      Yes      77.9         73.5         64.4
## 3                      4      Yes      53.5         38.3         36.3
## 4                      4      No       28.3         23.5         32.0
## 5                      0      No       74.7         54.9         73.6
## 6                      2      No       40.3         42.2         44.1
##   science_score final_exam_score previous_gpa pass_fail grade_category
## 1          51.8          49.1          2.44    Fail      F
## 2          61.6          70.1          2.79    Pass      C
## 3          47.1          42.2          1.49    Fail      F
## 4          39.0          31.9          1.34    Fail      F
## 5          55.5          66.4          2.60    Pass      C
## 6          30.0          40.9          1.61    Fail      F
```

```
str(dfi)
```

```
## 'data.frame':    10000 obs. of  23 variables:
##  $ student_id      : chr  "S00001" "S00002" "S00003" "S00004" ...
##  $ gender          : chr  "Male" "Female" "Male" "Male" ...
##  $ age             : int   17 18 17 18 18 17 16 18 17 15 ...
##  $ parental_education : chr  "High School" "High School" "High School" "Bachelor" ...
##  $ family_income    : chr  "Medium" "Low" "Medium" "Medium" ...
##  $ internet_access  : chr  "Yes" "Yes" "No" "Yes" ...
```

```
## $ study_environment      : chr "Quiet" "Quiet" "Quiet" "Quiet" ...
## $ study_hours_per_day    : num 2.98 4.45 3.75 2.03 5.14 1.16 3.33 2.01 1.22 1.64 ...
## $ attendance_rate        : num 96.5 95.7 76 72.6 87.3 92 74.2 88 70.4 95.6 ...
## $ sleep_hours            : num 6.05 6.96 7.02 6.23 8.54 8.16 7.34 6.58 7.96 7.51 ...
## $ social_media_hours     : num 0.1 2.9 2.4 3.5 2.1 2.6 3.9 0 1.1 1.7 ...
## $ assignment_completion_rate: num 80.5 70.9 77.6 63.5 71.8 100 81.3 87.3 79.6 81 ...
## $ participation_score    : num 68.7 92.6 45.8 72.9 55.7 76.3 41.8 74.9 99.1 61.8 ...
## $ online_courses_completed : int 1 0 4 4 0 2 1 4 2 2 ...
## $ tutoring               : chr "Yes" "Yes" "Yes" "No" ...
## $ math_score             : num 42.8 77.9 53.5 28.3 74.7 40.3 52.1 45.6 23.9 59.6 ...
## $ reading_score          : num 62.4 73.5 38.3 23.5 54.9 42.2 48.4 57.1 26.9 46.2 ...
## $ writing_score           : num 54.8 64.4 36.3 32 73.6 44.1 49.4 47.4 32.3 68 ...
## $ science_score          : num 51.8 61.6 47.1 39 55.5 30 47.8 67 32.8 48.3 ...
## $ final_exam_score       : num 49.1 70.1 42.2 31.9 66.4 40.9 47.1 57 30.2 54.8 ...
## $ previous_gpa           : num 2.44 2.79 1.49 1.34 2.6 1.61 1.35 2.52 1.2 2.02 ...
## $ pass_fail              : chr "Fail" "Pass" "Fail" "Fail" ...
## $ grade_category         : chr "F" "C" "F" "F" ...
```

Missing Values, Outlier Screening, and Final Dataset Formatting

Missing Values

```
# Check missing values
missing_counts <- colSums(is.na(dfi))
missing_counts
```

```
##          student_id          gender
##              0              0
##          age      parental_education
##              0              0
##      family_income      internet_access
##              0              0
##      study_environment      study_hours_per_day
##              0              0
##      attendance_rate      sleep_hours
##              0              0
##      social_media_hours      assignment_completion_rate
##              0              0
##      participation_score      online_courses_completed
##              0              0
##          tutoring          math_score
##              0              0
##      reading_score      writing_score
##              0              0
##      science_score      final_exam_score
##              0              0
##      previous_gpa      pass_fail
##              0              0
##      grade_category
##              0
```

```
sum(missing_counts)
```

```
## [1] 0
```

We first checked for missing values across all variables.

No missing values were detected.

No imputation was required.

No rows were removed at this stage.

Outlier Screening

Because several variables (scores, GPA, attendance, and completion rates) are naturally bounded by well-defined academic scales, range validation was considered sufficient for these features.

Extreme values within valid ranges (e.g., 0 or 100) were treated as plausible observations. These values were retained because they may represent very strong or very weak student performance. Additional outlier screening was focused only on less strictly bounded behavioral variables:

- study hours
- age
- sleep duration
- social media usage

```
# Check age range  
range(dfi$age, na.rm = TRUE)
```

```
## [1] 15 18
```

```
summary(dfi$age)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.  
##    15.00  15.00   16.00   16.49   17.00   18.00
```

```
# Variables selected for outlier screening  
outlier_vars <- c("study_hours_per_day", "sleep_hours", "social_media_hours")
```

```
outlier_summary <- data.frame(  
  Feature = character(),  
  Min = numeric(),  
  Max = numeric(),  
  stringsAsFactors = FALSE  
)  
  
for (col in outlier_vars) {  
  outlier_summary <- rbind(  
    outlier_summary,  
    data.frame(  
      Feature = col,  
      Min = min(dfi[[col]], na.rm = TRUE),  
      Max = max(dfi[[col]], na.rm = TRUE)  
    )  
  )  
}
```

```

    )
  }

outlier_summary

##           Feature Min   Max
## 1 study_hours_per_day 0.5  7.24
## 2           sleep_hours 4.0 10.00
## 3  social_media_hours 0.0  8.00

# Check discrete count variable separately
range(df1$online_courses_completed, na.rm = TRUE)

```

```
## [1] 0 9
```

```
table(df1$online_courses_completed)
```

```
##
##    0    1    2    3    4    5    6    7    8    9
## 1341 2687 2645 1821  915  414  128  40   6   3
```

Dataset Formatting

At this stage, variables were reviewed and formatted to ensure consistency before modeling.

Final Dataset

The final modeling dataset was created after removing variables with no predictive value or variables that could introduce data leakage.

Removed Variables:

student_id → unique identifier with no predictive meaning

age → unrealistic range relative to academic levels represented

final_exam_score → directly tied to pass/fail outcome previous_gpa → highly reflective of academic performance

grade_category → directly encodes final academic outcome

Removing these variables helps ensure a fair and realistic predictive task.

```

# Final modeling dataset
library(tidyverse)

```

```

## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.1      v stringr    1.5.2
## v ggplot2    4.0.1      v tibble     3.2.1
## v lubridate  1.9.4      v tidyr      1.3.1
## v purrr      1.0.4
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors

```

```
df_model <- dfi %>%
  select(
    -student_id,
    -age,
    -final_exam_score,
    -previous_gpa,
    -grade_category
  )

# Convert target and categorical variables
df_model$pass_fail <- as.factor(df_model$pass_fail)

df_model <- df_model %>%
  mutate(across(where(is.character), as.factor))

# Final check
dim(df_model)
```

```
## [1] 10000    18
```

```
head(df_model)
```

```
##   gender parental_education family_income internet_access study_environment
## 1   Male      High School      Medium      Yes           Quiet
## 2 Female      High School      Low        Yes           Quiet
## 3   Male      High School      Medium      No            Quiet
## 4   Male      Bachelor      Medium      Yes           Quiet
## 5   Male      Bachelor      Medium      Yes           Quiet
## 6 Female      Master       Medium      Yes           Quiet
##   study_hours_per_day attendance_rate sleep_hours social_media_hours
## 1             2.98           96.5         6.05             0.1
## 2             4.45           95.7         6.96             2.9
## 3             3.75           76.0         7.02             2.4
## 4             2.03           72.6         6.23             3.5
## 5             5.14           87.3         8.54             2.1
## 6             1.16           92.0         8.16             2.6
##   assignment_completion_rate participation_score online_courses_completed
## 1             80.5              68.7              1
## 2             70.9              92.6              0
## 3             77.6              45.8              4
## 4             63.5              72.9              4
## 5             71.8              55.7              0
## 6            100.0              76.3              2
##   tutoring math_score reading_score writing_score science_score pass_fail
## 1     Yes      42.8       62.4         54.8         51.8      Fail
## 2     Yes      77.9       73.5         64.4         61.6      Pass
## 3     Yes      53.5       38.3         36.3         47.1      Fail
## 4     No       28.3       23.5         32.0         39.0      Fail
## 5     No       74.7       54.9         73.6         55.5      Pass
## 6     No       40.3       42.2         44.1         30.0      Fail
```

Final Formatting Steps

The target variable `pass_fail` was converted to a factor.
All character-based predictors were converted to factors.
This ensures compatibility with Logistic Regression and Random Forest

EDA

Why These Visualizations?

The selected graphs were chosen to directly support the pass/fail prediction objective.

Selected Visualizations:

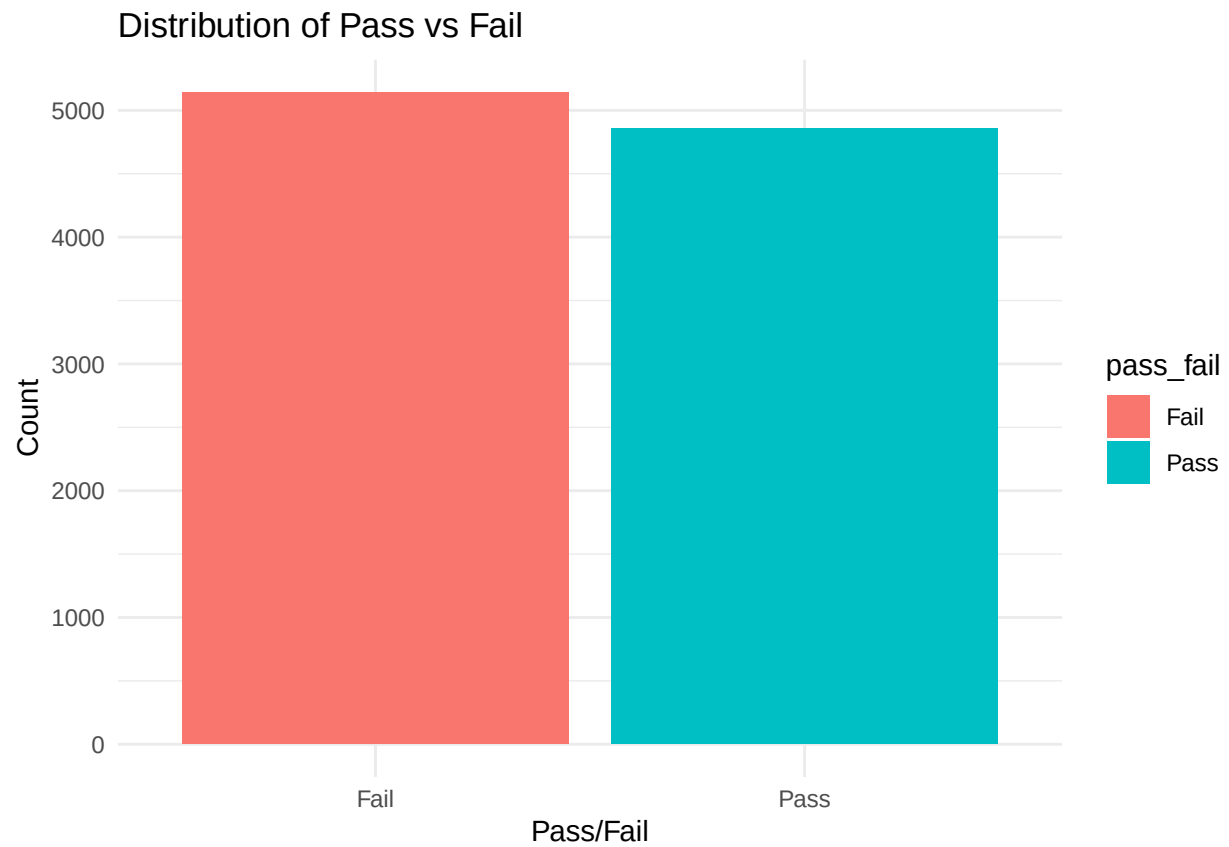
1. Target distribution plot
 - checks whether the classes are balanced before modeling
2. Boxplots of key numerical variables
 - compare study behavior and academic scores between pass and fail groups
3. Proportional bar plots for categorical variables
 - assess whether factors such as tutoring and internet access are associated with pass rates
4. Correlation heatmap
 - identifies relationships and possible redundancy among numerical predictors (especially useful for Logistic Regression)

These visualizations help identify which variables appear most informative before model training.

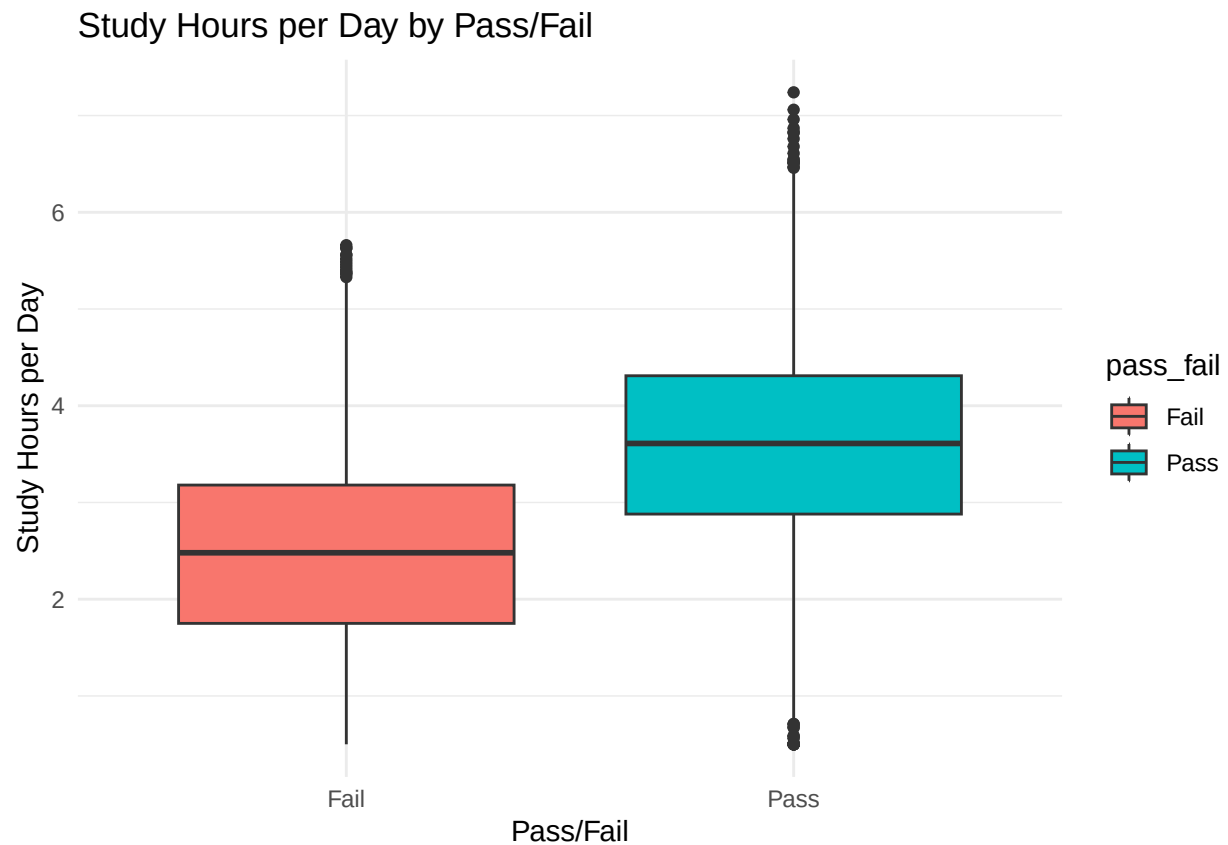
```
library(ggplot2)
library(tidyverse)
library(corrplot)
```

```
## corrplot 0.95 loaded
```

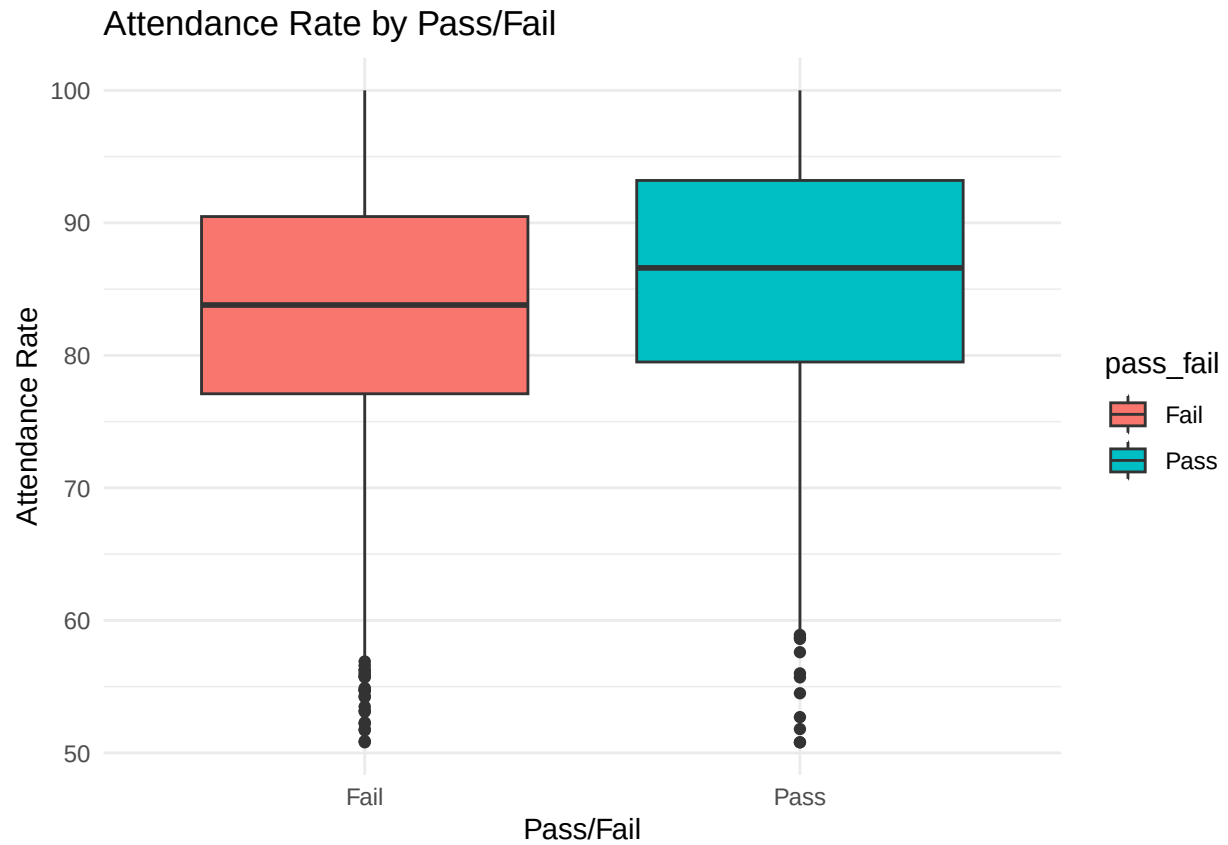
```
ggplot(df_model, aes(x = pass_fail, fill = pass_fail)) +
  geom_bar() +
  labs(
    title = "Distribution of Pass vs Fail",
    x = "Pass/Fail",
    y = "Count"
  ) +
  theme_minimal()
```



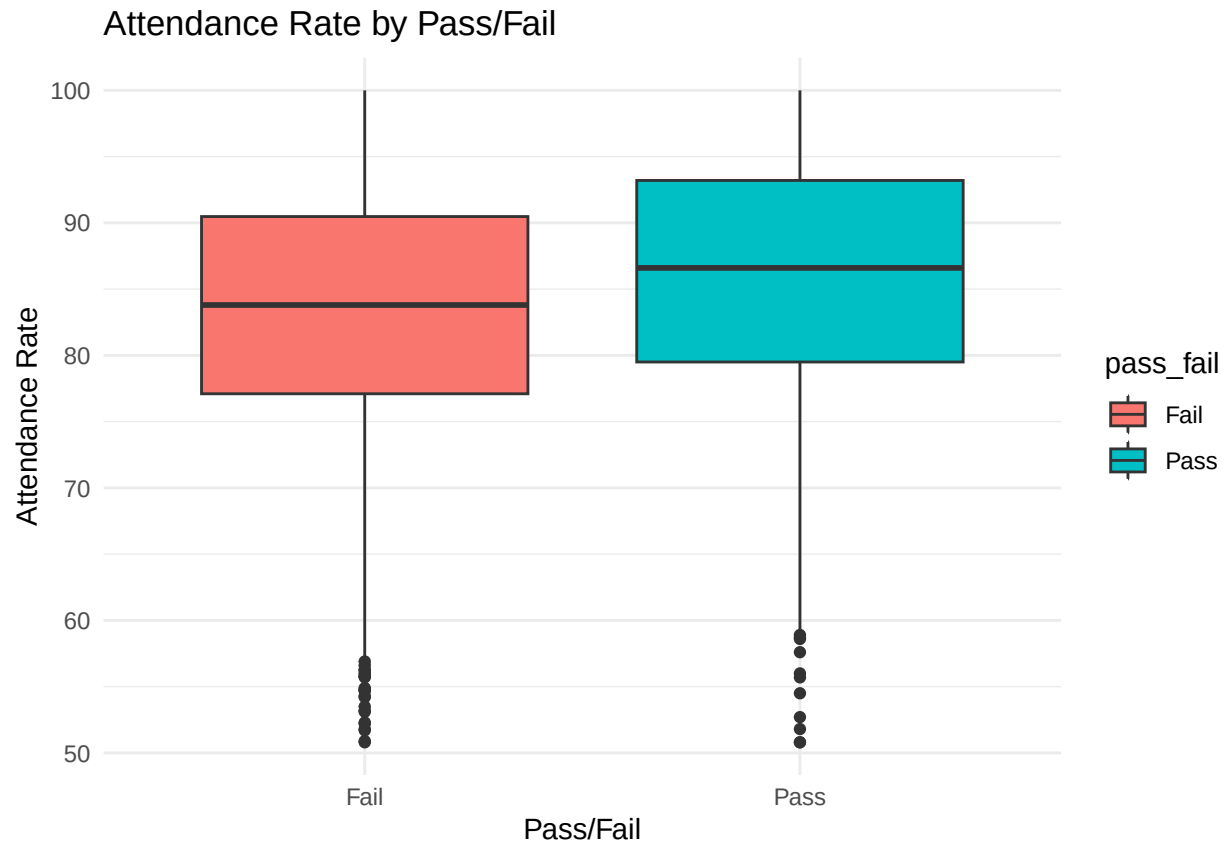
```
ggplot(df_model, aes(x = pass_fail, y = study_hours_per_day, fill = pass_fail)) +  
  geom_boxplot() +  
  labs(  
    title = "Study Hours per Day by Pass/Fail",  
    x = "Pass/Fail",  
    y = "Study Hours per Day"  
  ) +  
  theme_minimal()
```

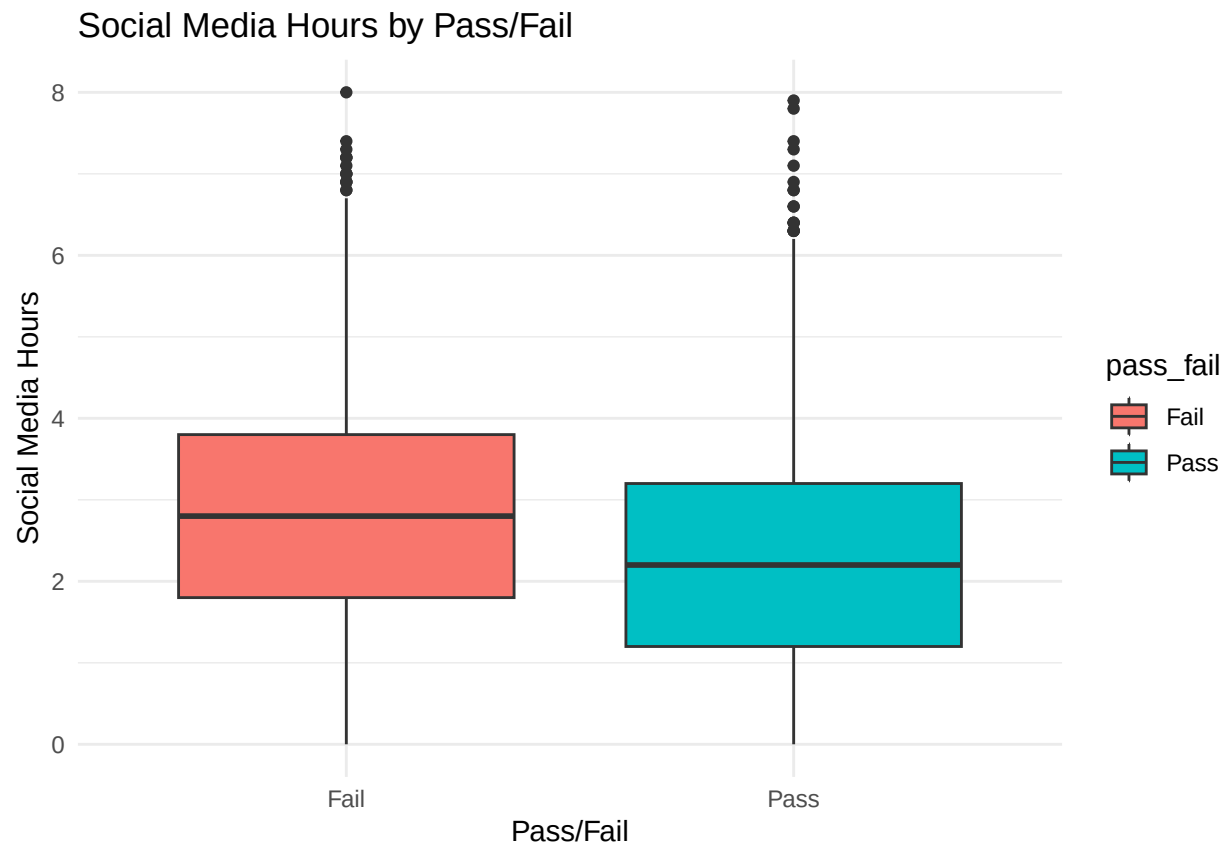
```
ggplot(df_model, aes(x = pass_fail, y = attendance_rate, fill = pass_fail)) +  
  geom_boxplot() +  
  labs(  
    title = "Attendance Rate by Pass/Fail",  
    x = "Pass/Fail",  
    y = "Attendance Rate"  
  ) +  
  theme_minimal()
```



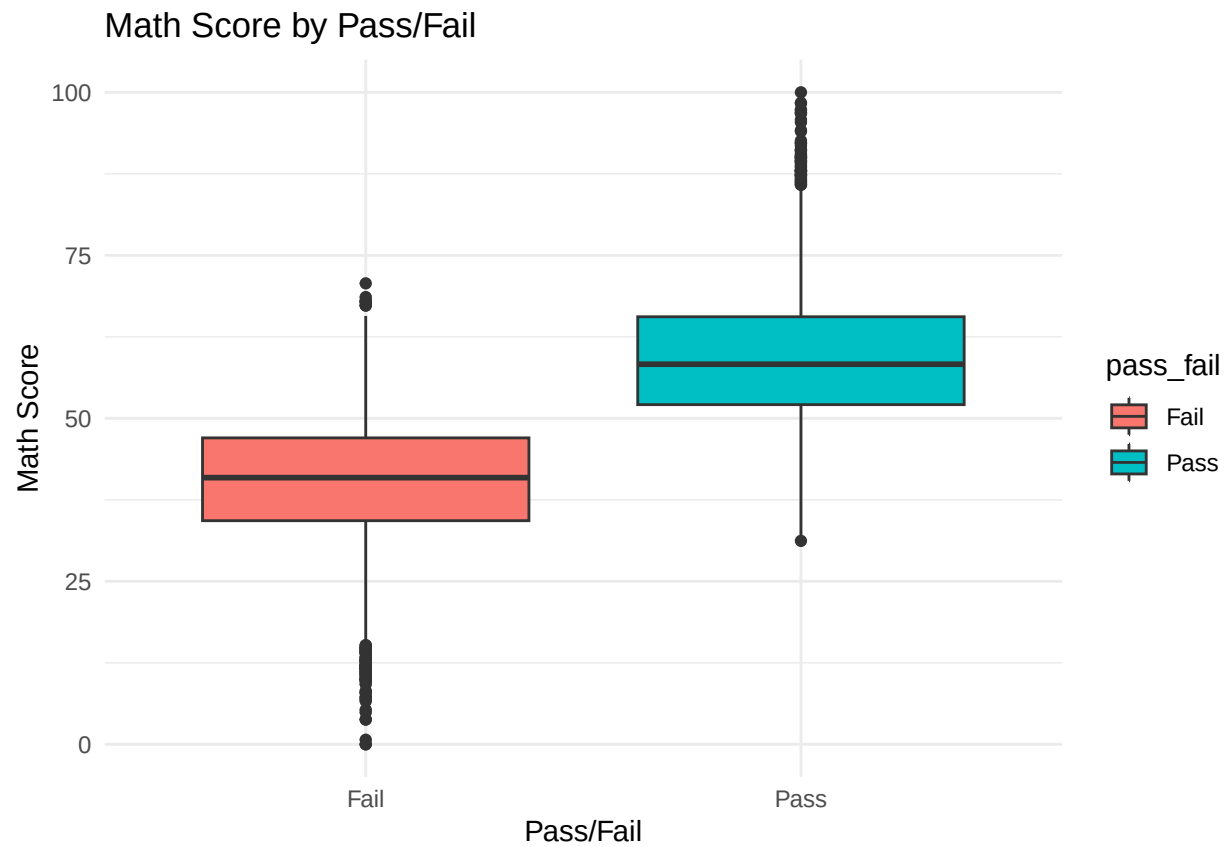
```
ggplot(df_model, aes(x = pass_fail, y = attendance_rate, fill = pass_fail)) +  
  geom_boxplot() +  
  labs(  
    title = "Attendance Rate by Pass/Fail",  
    x = "Pass/Fail",  
    y = "Attendance Rate"  
  ) +  
  theme_minimal()
```



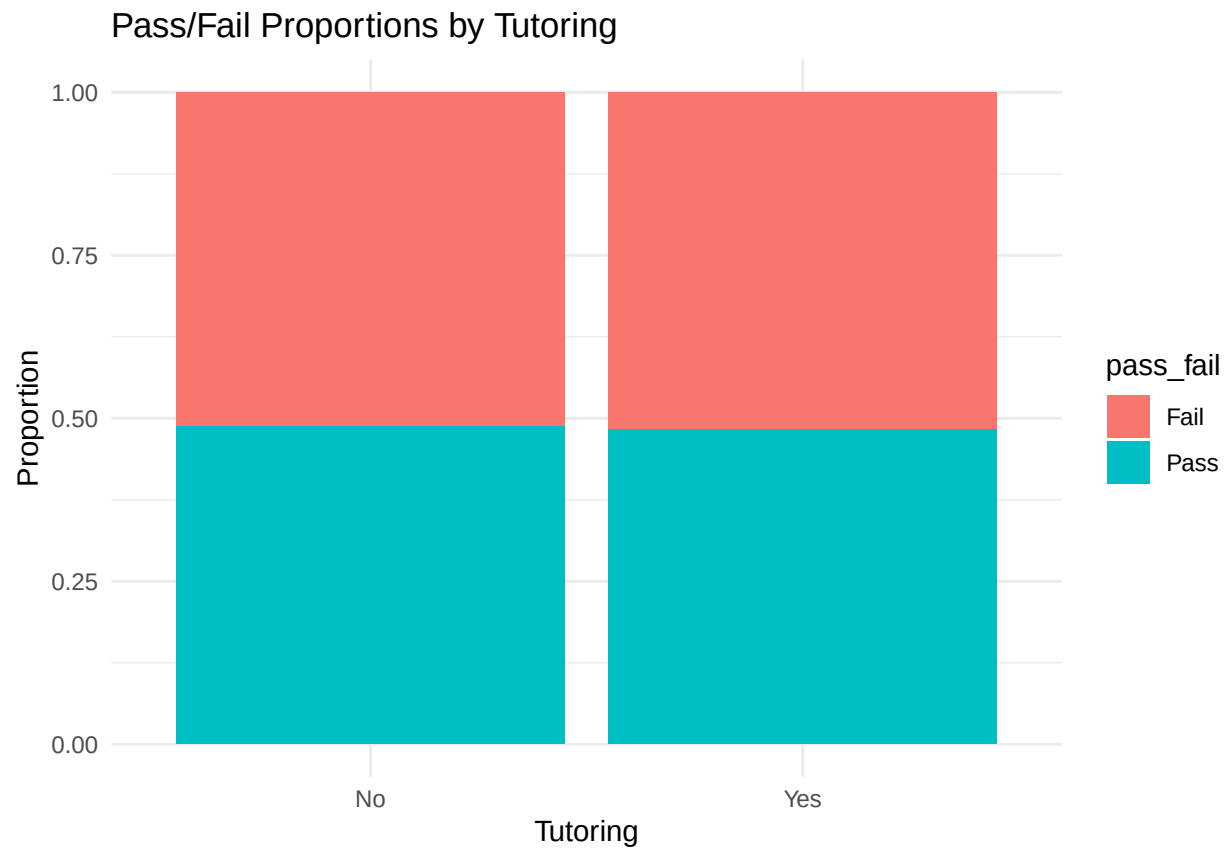
```
ggplot(df_model, aes(x = pass_fail, y = social_media_hours, fill = pass_fail)) +  
  geom_boxplot() +  
  labs(  
    title = "Social Media Hours by Pass/Fail",  
    x = "Pass/Fail",  
    y = "Social Media Hours"  
  ) +  
  theme_minimal()
```



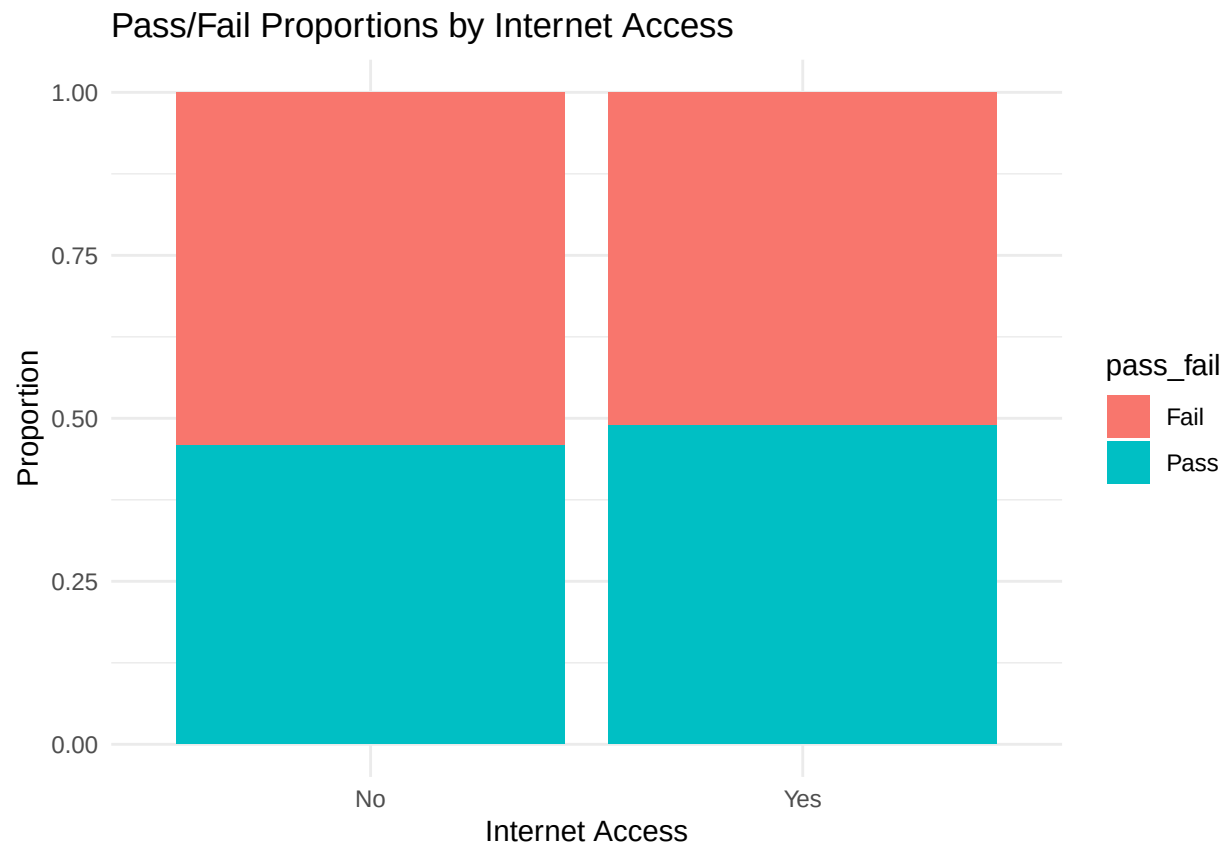
```
ggplot(df_model, aes(x = pass_fail, y = math_score, fill = pass_fail)) +  
  geom_boxplot() +  
  labs(  
    title = "Math Score by Pass/Fail",  
    x = "Pass/Fail",  
    y = "Math Score"  
  ) +  
  theme_minimal()
```



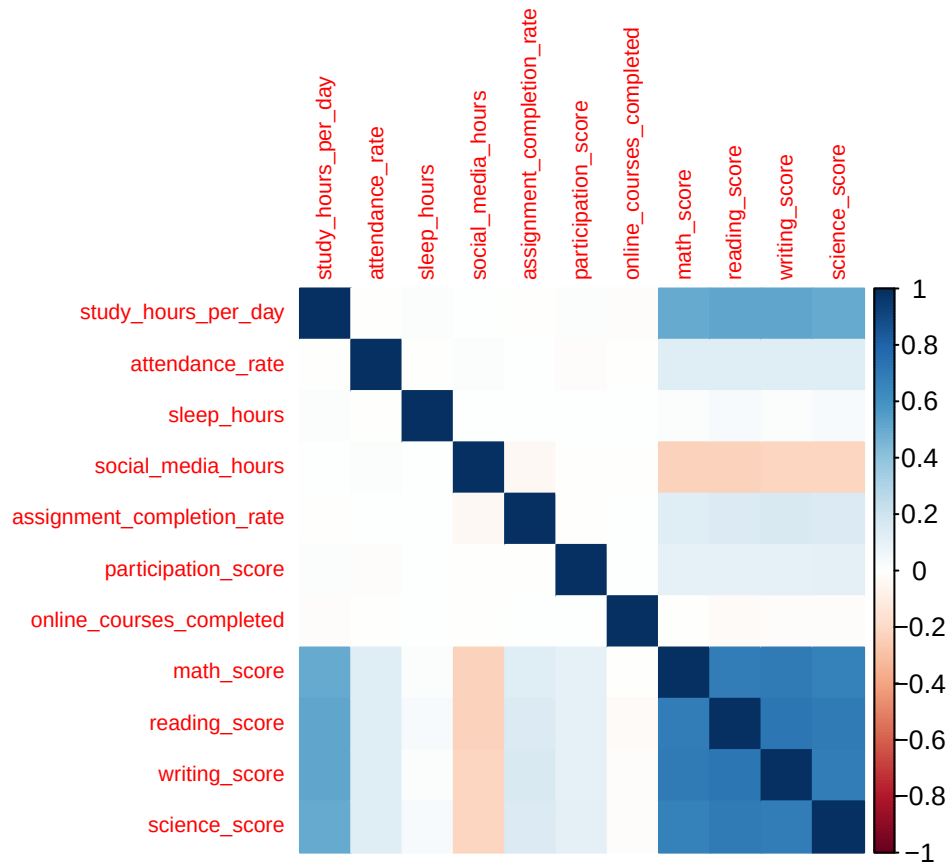
```
ggplot(df_model, aes(x = tutoring, fill = pass_fail)) +  
  geom_bar(position = "fill") +  
  labs(  
    title = "Pass/Fail Proportions by Tutoring",  
    x = "Tutoring",  
    y = "Proportion"  
  ) +  
  theme_minimal()
```



```
ggplot(df_model, aes(x = internet_access, fill = pass_fail)) +  
  geom_bar(position = "fill") +  
  labs(  
    title = "Pass/Fail Proportions by Internet Access",  
    x = "Internet Access",  
    y = "Proportion"  
  ) +  
  theme_minimal()
```



```
numeric_df <- df_model %>%  
  select(where(is.numeric))  
  
cor_matrix <- cor(numeric_df)  
  
corrplot(cor_matrix, method = "color", tl.cex = 0.7)
```



Main Findings and Next Steps

The EDA suggests that the dataset contains strong predictive signal.

Key observations:

The target classes are well balanced.

Students who pass have higher study hours per day.

Attendance is slightly higher among passing students.

Social media use tends to be lower among passing students.

Math score shows the clearest separation, making it a likely strong predictor.

Tutoring and internet access show weaker isolated effects.

Subject scores are strongly correlated, suggesting some redundancy among academic variables.

Train/Test Split

A stratified 75/25 train-test split was used to preserve the pass/fail class proportions while maintaining random sampling for fair model comparison.

```
library(caret)
```

```
## Loading required package: lattice
```

```
##
```

```
## Attaching package: 'caret'
```

```
## The following object is masked from 'package:purrr':
```



```
##
## lift

set.seed(42)

#Train/Test Split
train_index <- createDataPartition(df_model$pass_fail, p = 0.75, list = FALSE)

train_data <- df_model[train_index, ]
test_data <- df_model[-train_index, ]
prop.table(table(train_data$pass_fail))

##
##      Fail      Pass
## 0.5141981 0.4858019

prop.table(table(test_data$pass_fail))

##
##      Fail      Pass
## 0.5142057 0.4857943

#Logistic Regression Data Preparation
# Create dummy variables
dummies <- dummyVars(pass_fail ~ ., data = train_data)

x_train_log <- predict(dummies, newdata = train_data)

## Warning in model.frame.default(Terms, newdata, na.action = na.action, xlev =
## object$lvls): variable 'pass_fail' is not a factor

x_test_log <- predict(dummies, newdata = test_data)

## Warning in model.frame.default(Terms, newdata, na.action = na.action, xlev =
## object$lvls): variable 'pass_fail' is not a factor

# Scale numeric predictors
preProcValues <- preProcess(x_train_log, method = c("center", "scale"))

x_train_log <- predict(preProcValues, x_train_log)
x_test_log <- predict(preProcValues, x_test_log)

y_train <- train_data$pass_fail
y_test <- test_data$pass_fail

#Random Forest Data Preparation
x_train_rf <- train_data
x_test_rf <- test_data
```

For Logistic Regression, categorical predictors must be converted into dummy variables, and numerical predictors must be standardized so that regularization treats all features on a comparable scale.

Model 1 — Regularized Logistic Regression with tuning

A 10-fold cross-validation procedure was used on the training set to tune the regularization parameter. The same random seed was maintained throughout the workflow to ensure reproducibility of both the train-test split and the cross-validation folds.

```
library(glmnet)
```

```
## Loading required package: Matrix
```

```
##
```

```
## Attaching package: 'Matrix'
```

```
## The following objects are masked from 'package:tidyr':
```

```
##
```

```
##      expand, pack, unpack
```

```
## Loaded glmnet 4.1-8
```

```
library(caret)
```

```
# Convert target into 0/1
```

```
y_train_num <- ifelse(y_train == "Pass", 1, 0)
```

```
y_test_num  <- ifelse(y_test  == "Pass", 1, 0)
```

```
# Convert X into matrix
```

```
x_train_mat <- as.matrix(x_train_log)
```

```
x_test_mat  <- as.matrix(x_test_log)
```

```
#Tuning with cross validation
```

```
set.seed(42)
```

```
cv_model <- cv.glmnet(
```

```
  x_train_mat,
```

```
  y_train_num,
```

```
  family = "binomial",
```

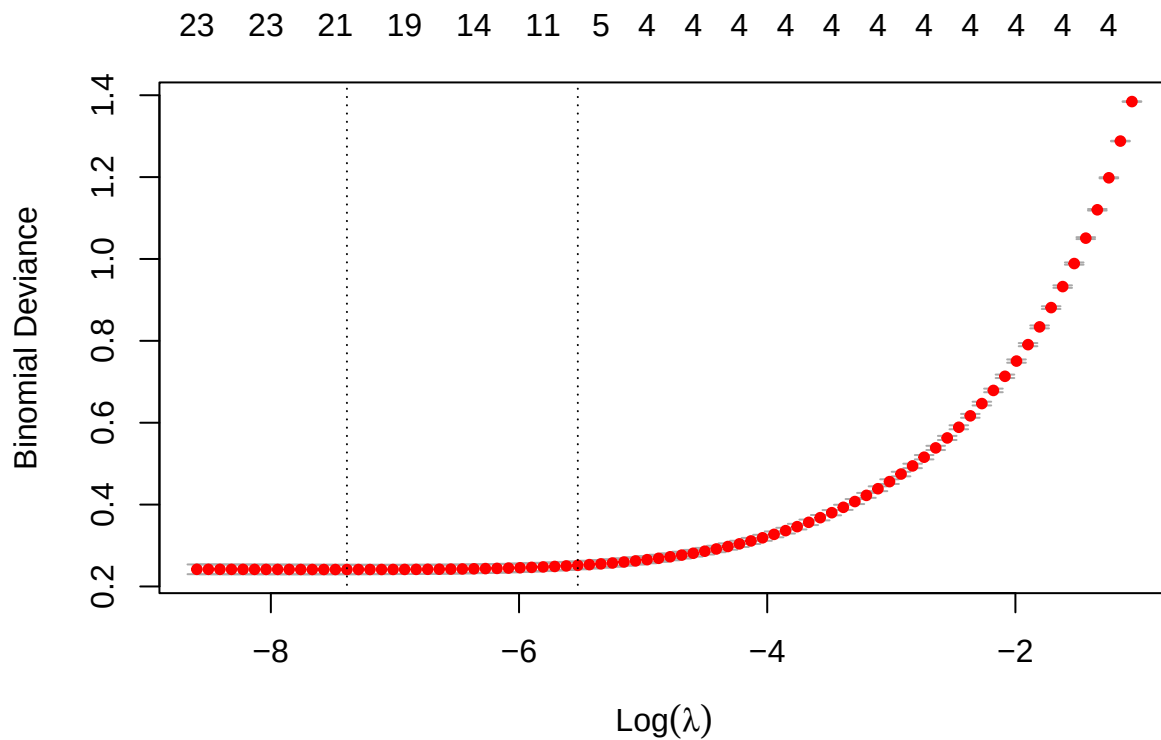
```
  alpha = 1,
```

```
  nfolds = 10
```

```
)
```

```
#Tuning visualization
```

```
plot(cv_model)
```



```
#Best lambda
best_lambda <- cv_model$lambda.min
best_lambda
```

```
## [1] 0.0006192374
```

```
#Training with best lambda(Final model)
log_model <- glmnet(
  x_train_mat,
  y_train_num,
  family = "binomial",
  alpha = 1,
  lambda = best_lambda
)

#Prediction
# Probabilities
prob_pred <- predict(log_model, newx = x_test_mat, type = "response")

# Conversion to binary class labels (Threshold 0.5)
pred_class <- ifelse(prob_pred > 0.5, 1, 0)

#Model Evaluation
confusionMatrix(
  factor(pred_class),
```

```
factor(y_test_num)
)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##           0 1209   65
##           1   76 1149
##
##           Accuracy : 0.9436
##           95% CI : (0.9338, 0.9523)
##       No Information Rate : 0.5142
##       P-Value [Acc > NIR] : <2e-16
##
##           Kappa : 0.8871
##
##  McNemar's Test P-Value : 0.3997
##
##           Sensitivity : 0.9409
##           Specificity : 0.9465
##           Pos Pred Value : 0.9490
##           Neg Pred Value : 0.9380
##           Prevalence : 0.5142
##           Detection Rate : 0.4838
##       Detection Prevalence : 0.5098
##           Balanced Accuracy : 0.9437
##
##           'Positive' Class : 0
##
```

```
#ROC Curve
library(pROC)
```

```
## Type 'citation("pROC")' for a citation.
```

```
##
## Attaching package: 'pROC'
```

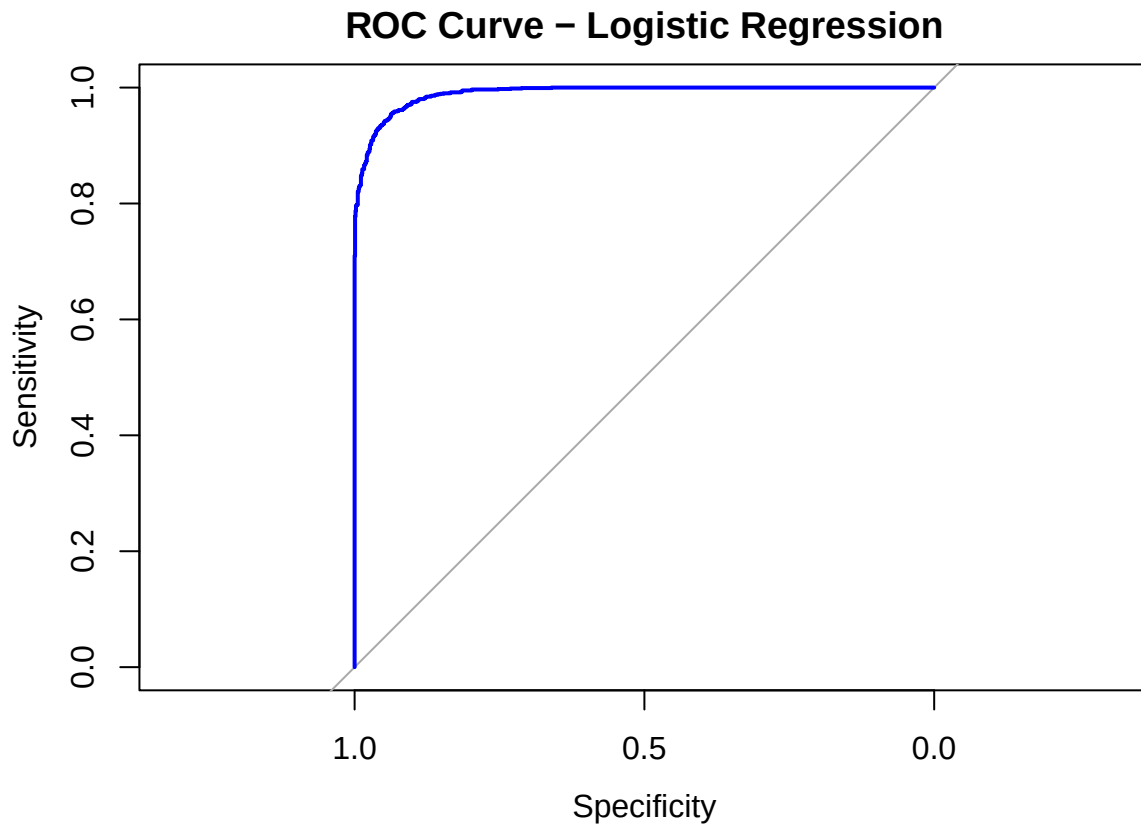
```
## The following objects are masked from 'package:stats':
##
##     cov, smooth, var
```

```
roc_obj <- roc(y_test_num, as.numeric(prob_pred))
```

```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls < cases
```

```
plot(roc_obj, col = "blue", main = "ROC Curve - Logistic Regression")
```



```
auc(roc_obj)
```

```
## Area under the curve: 0.9904
```

The LASSO-regularized Logistic Regression model achieved strong predictive performance on the test set, with an accuracy of 94.36% and a balanced accuracy of 94.37%. Both sensitivity (94.09%) and specificity (94.65%) indicate that the model performs well in identifying both failing and passing students without bias toward a specific class.

The ROC curve further confirms the model's effectiveness, with an AUC of 0.9904, indicating excellent discrimination ability between the two classes. These results suggest that the selected predictors contain strong predictive signal for academic success classification.

However, the very high performance may also indicate that some predictors, particularly subject scores, are highly correlated with the outcome variable, making the classification task relatively easy.

Model 2 Random Forest with Hyperparameter Tuning

A Random Forest classifier was trained on the same stratified train-test split. Hyperparameter tuning focused on the number of variables randomly sampled at each split (`mtry`) to optimize classification performance.

```

library(randomForest)

## randomForest 4.7-1.2

## Type rfNews() to see new features/changes/bug fixes.

##
## Attaching package: 'randomForest'

## The following object is masked from 'package:dplyr':
##
##      combine

## The following object is masked from 'package:ggplot2':
##
##      margin

library(caret)
library(pROC)

# Use split data directly for RF
train_rf <- train_data
test_rf  <- test_data

# Hyperparameter tuning
set.seed(42)

rf_tuned <- train(
  pass_fail ~ .,
  data = train_rf,
  method = "rf",
  trControl = trainControl(method = "cv", number = 10),
  tuneLength = 5
)

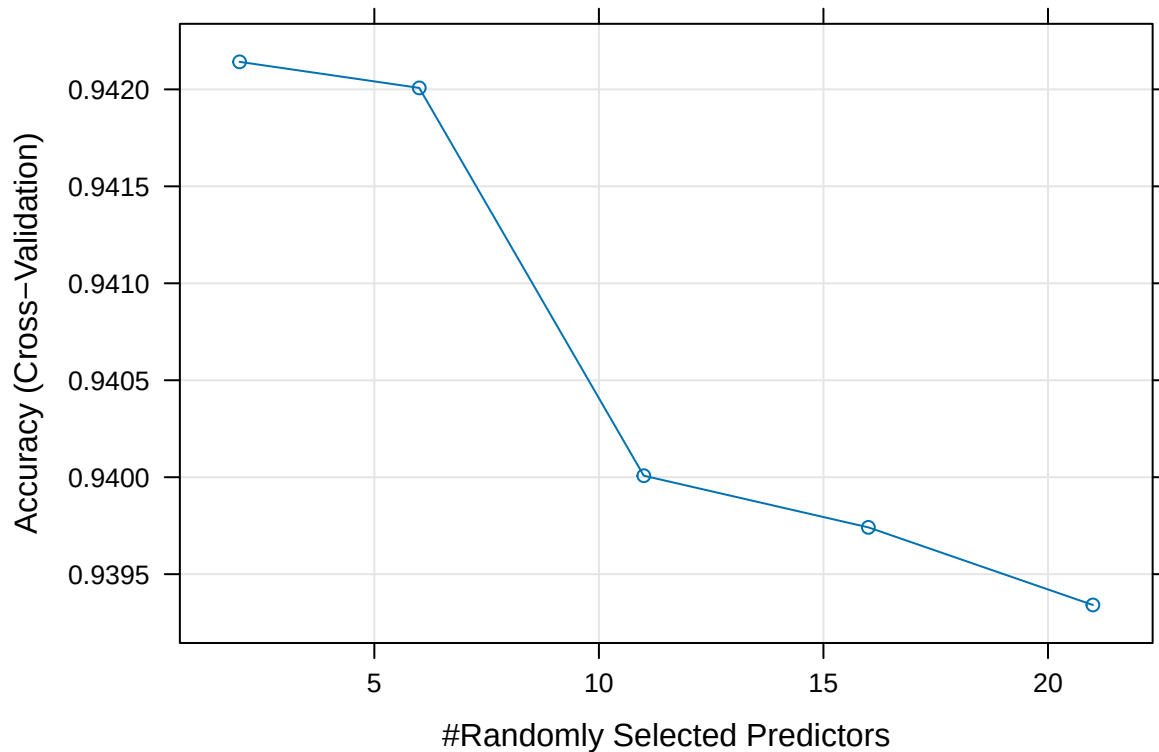
rf_tuned

## Random Forest
##
## 7501 samples
## 17 predictor
## 2 classes: 'Fail', 'Pass'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 6751, 6751, 6751, 6751, 6750, 6751, ...
## Resampling results across tuning parameters:
##
## mtry Accuracy Kappa
## 2 0.9421420 0.8841714
## 6 0.9420076 0.8839142

```

```
## 11 0.9400076 0.8799115
## 16 0.9397413 0.8793776
## 21 0.9393411 0.8785800
##
## Accuracy was used to select the optimal model using the largest value.
## The final value used for the model was mtry = 2.
```

```
plot(rf_tuned)
```



```
# Best mtry
best_mtry <- rf_tuned$bestTune$mtry
best_mtry
```

```
## [1] 2
```

```
# Final model
set.seed(42)

rf_model <- randomForest(
  pass_fail ~ .,
  data = train_rf,
  mtry = best_mtry,
  importance = TRUE
)
```

```

# Prediction
rf_pred <- predict(rf_model, newdata = test_rf)

# Confusion matrix
rf_cm <- confusionMatrix(rf_pred, test_rf$pass_fail)
rf_cm

## Confusion Matrix and Statistics
##
##              Reference
## Prediction Fail Pass
##      Fail 1208   65
##      Pass   77 1149
##
##              Accuracy : 0.9432
##              95% CI : (0.9334, 0.9519)
##      No Information Rate : 0.5142
##      P-Value [Acc > NIR] : <2e-16
##
##              Kappa : 0.8863
##
##  Mcnemar's Test P-Value : 0.356
##
##              Sensitivity : 0.9401
##              Specificity : 0.9465
##      Pos Pred Value : 0.9489
##      Neg Pred Value : 0.9372
##      Prevalence : 0.5142
##      Detection Rate : 0.4834
##      Detection Prevalence : 0.5094
##      Balanced Accuracy : 0.9433
##
##      'Positive' Class : Fail
##

# ROC + AUC
rf_prob <- predict(rf_model, newdata = test_rf, type = "prob")[, "Pass"]

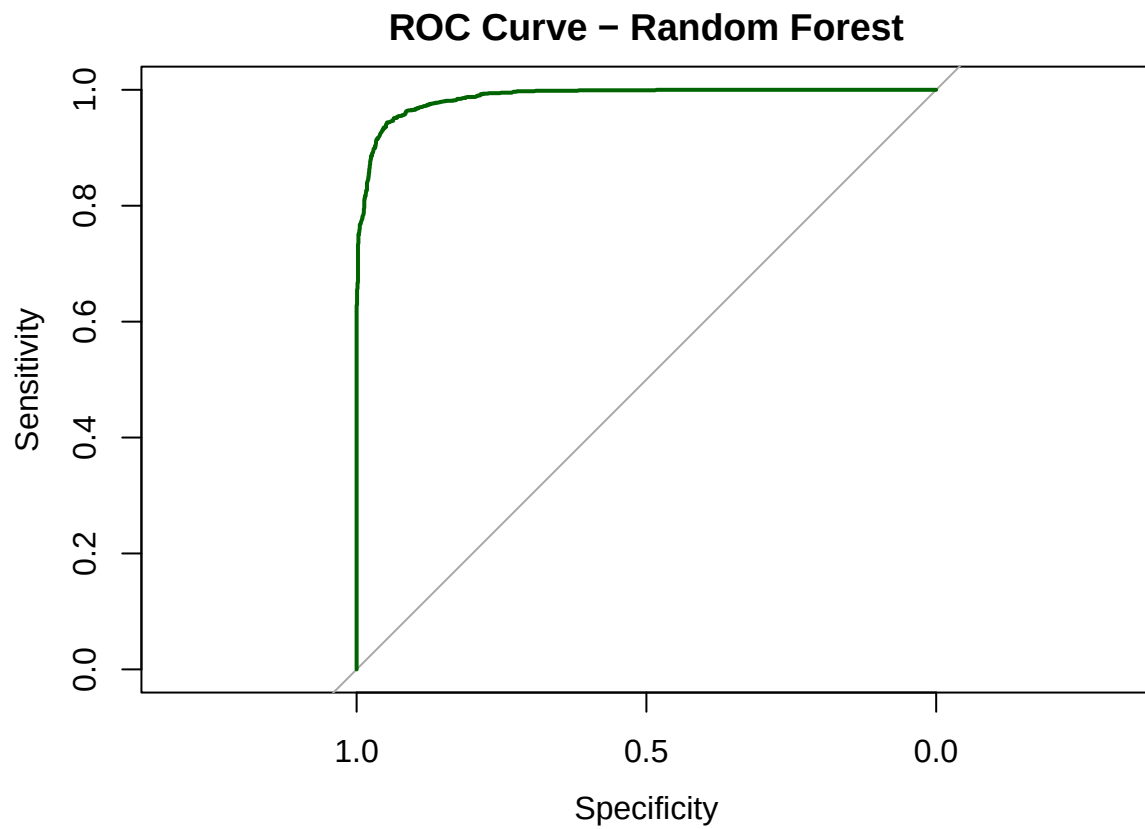
rf_roc <- roc(test_rf$pass_fail, rf_prob)

## Setting levels: control = Fail, case = Pass

## Setting direction: controls < cases

plot(rf_roc, col = "darkgreen", main = "ROC Curve - Random Forest")

```

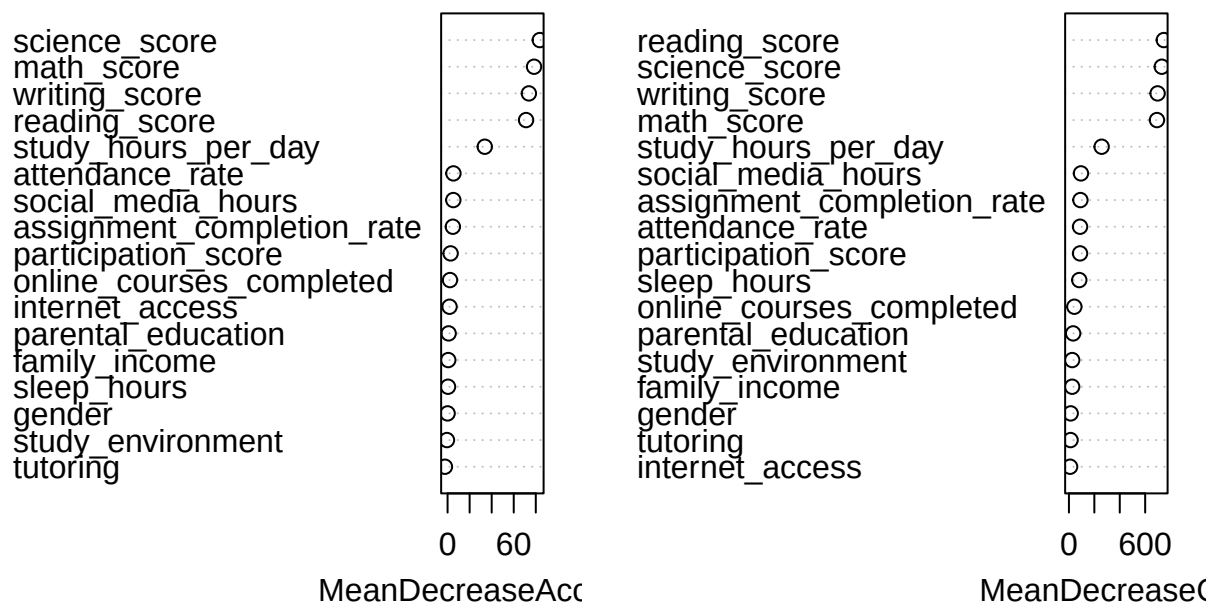



```
auc(rf_roc)
```

```
## Area under the curve: 0.9875
```

```
# Variable importance  
varImpPlot(rf_model)
```

rf_model



The tuned Random Forest model achieved excellent classification performance, with an accuracy of 94.32%, a balanced accuracy of 94.33%, and an AUC of 0.9875. The optimal hyperparameter value was `mtry = 2`, indicating that considering a small number of predictors at each split improved tree diversity and overall generalization performance.

The variable importance analysis showed that academic subject scores—particularly reading, science, writing, and math scores—were the strongest predictors of pass/fail outcomes. Behavioral variables such as study hours per day also contributed meaningfully, while contextual factors such as tutoring and internet access had weaker isolated influence.

Overall, the Random Forest model performed extremely well and confirmed that both academic performance indicators and selected study behaviors contain strong predictive signal for student success classification.

Conclusion

This project compared LASSO-regularized Logistic Regression and Random Forest for predicting student pass/fail outcomes using behavioral, contextual, and academic predictors.

Both models achieved excellent and nearly identical performance, with test accuracies above 94% and AUC values close to 0.99, indicating outstanding discriminative ability. While Random Forest effectively captured non-linear relationships and confirmed the importance of academic subject scores, Logistic Regression delivered slightly better performance while offering significantly greater interpretability.

Because the predictive performance difference between the two models is minimal, Logistic Regression is selected as the preferred final model due to its simplicity, transparency, and ease of interpretation. The results also suggest that subject-level academic scores and study habits are the most influential factors in determining student success, with math, reading, science, writing, and study hours emerging as the strongest predictors.

One important limitation of this study is that the very high predictive performance may be partly driven by the inclusion of subject-level academic scores, which are strongly related to the final pass/fail outcome. This makes the classification task relatively easy and may limit generalization to settings where only earlier behavioral or contextual predictors are available.